Project Proposal and Requirements Analysis
APPDAET - Group 6: Numerical Root Finder

A. Project Understanding and Scope

A1. Introduce the assigned final project and explain its purpose.

Our assigned final project is a Numerical Root Finder, a Python desktop application that will approximate one real root of a third-degree or cubic polynomial using the Bisection Method. Users will be able to choose a built-in equation or enter their own coefficients, then set the interval, tolerance, and maximum number of iterations.

The purpose of the application is to make the calculation easier to perform and understand. It will show the estimated root, the calculations from each iteration, and a graph so users can see how the method approaches a solution.

A2. Identify the problem the application intends to solve.

Finding a root of a cubic equation manually can involve repeated calculations, especially when the root is not a whole number. Keeping track of the changing interval and checking when to stop can take time and lead to calculation errors.

Our application will address this by performing the repeated calculations using the Bisection Method. It will keep a record of each iteration and explain why the calculation stopped. It will also check the inputs and notify the user if the selected interval does not meet the method's requirements.

A3. Enumerate the mandatory features identified in the Project Specification Sheet.

The application must include the following features:

1. **Equation selection.** Provide at least three built-in cubic equations and an option to enter coefficients manually.
2. **Coefficient entry.** Accept numerical values for a, b, c, and d, where a must not be zero.
3. **Root-search controls.** Allow users to enter the lower bound, upper bound, positive tolerance, and maximum number of iterations.
4. **Bisection algorithm.** Use our own Python functions to calculate one bracketed real root.

5. **Iteration table.** Show the iteration number, lower bound, upper bound, midpoint, and function value at the midpoint for every iteration.
6. **Polynomial graph.** Display the selected cubic equation and mark the current or final root position.
7. **Result display.** Show the approximate root, residual |f(x)|, iteration count, and reason for stopping.

The application must also handle invalid inputs without crashing and recognize when an endpoint is already a root.

A4. Identify the expected inputs, processing operations, and outputs.

Inputs

The user will select a preset equation or enter the coefficients a, b, c, and d. They will also provide the lower bound L, upper bound U, tolerance, and maximum number of iterations.

Processing operations

The application will first check whether the inputs are valid. This includes making sure that $a \neq 0$, $L < U$, the tolerance is positive, and the iteration limit is a positive integer.

It will then evaluate the function at both endpoints. If either endpoint is a root, the application will return that value. Otherwise, the endpoint values must have opposite signs before Bisection can proceed.

During each iteration, the application will calculate the midpoint, evaluate the function at that point, and keep the half-interval that brackets a root. It will record each step and continue until the stopping condition is met or the maximum number of iterations is reached. The exact stopping rule will be stated in the algorithm documentation.

Outputs

The application will display:

- The approximate root.
- The residual |f(x)|, showing how close the function value is to zero.
- The number of iterations performed.
- The reason the calculation stopped.
- The complete iteration table.
- The polynomial graph with the root position marked.
- A clear message when an input or bracket is invalid.

A5. Discuss the limitations and boundaries of the project.

Our application will focus on cubic polynomials in the form $f(x) = ax^3 + bx^2 + cx + d$, where $a \neq 0$. Equations outside this form are not part of the required scope.

The application is only required to find one bracketed real root. It will not be required to find every real root or any complex roots. Bisection normally requires opposite signs at the interval endpoints, unless an endpoint is already a root. If both endpoint values have the same nonzero sign, the application will reject the interval. This does not mean that the equation has no real root.

The result will be a numerical approximation. Its accuracy will depend on the selected tolerance, stopping rule, and iteration limit. If the iteration limit is reached first, the application will clearly report that the requested stopping condition has not been met.